```
forward = (callback) ->
  console.log 'moved forward'
  setTimeout callback, 200
```

How do you compose these asynchronous functions? The standard `compose` doesn't work because it expects to use the evaluation of one function as the argument to another:

```
startThenForward = compose forward, start
startThenForward (res) ->
  console.log res
# TypeError: undefined is not a function
```

You need a different `composeAsync` for asynchronous functions. Take a deep breath.

```
composeAsync = (f, g) -> (x) -> g -> f x
```

This async version of compose looks even scarier. Again, though, it's the classic way to express this combinator, so it's best if you learn to get used to it.

This version works, although it's limited to functions that don't take any arguments other than a single callback:

```
startThenForward = composeAsync forward, start
startThenForward ->
  console.log 'done'
# started
# moved forward
# done
```

When the asynchronous functions have arguments other than a callback, things become more interesting. In the next listing you see asynchronous versions of `before` and `after` that Agtron has provided. Take an even deeper breath and then spend some time experimenting with them and dissecting them so that you begin to understand how they work.

Listing 6.7. Asynchronous `before` and `after`

```
beforeAsync = (decoration) ->
  (base) ->
    (params..., callback) ->
      result = undefined
```